# 배지혁

신입 백엔드 개발자, 2026.02 졸업

qowlgur121@gmail.com / github.com/qowlgur121 / 포트폴리오 qowlgur121.github.io

## AI를 실제 업무에 붙이고, 돈이 걸린 처리는 권한과 검증으로 막습니다.



**대표 경험**

### 판매 운영 자동화, AI 운영

개인, 2026.09 ~ 10

Python, SQLite, Oracle Cloud

온라인 판매 운영을 혼자 맡을 수 있게 만든 주문 자동화 시스템. 결제 확인부터 발주와 발송까지 정해진 코드로 처리하고 AI 점검 에이전트에는 허용한 조치만 주어 돈이 걸린 결정은 사람이 내리게 함

**하루 최대 1,378건 실거래 처리**

### 공공기관 AI 상담 챗봇

3인 팀장, 2025.09 ~ 12

FastAPI, LangChain, pgvector

재학습이 필요하던 파인튜닝 챗봇을 매일 갱신되는 RAG로 재구축. LLM 의도 분류와 조건 필터로 신청 가능한 사업부터 안내하고 대화는 24시간 뒤 자동 삭제

**사업 1,809건, 시설 18,319곳, 캡스톤 우수상, 기관 표창**

### OpenContext

3인 팀, 2025.08 ~ 12

Spring Boot, Elasticsearch, MCP

사내 문서의 저장과 임베딩과 검색을 조직 서버 안에서 처리하는 MCP 검색 서버를 제안하고 설계. 하이브리드 검색과 2단계 조회 API(목록 → 고른 원문만) 담당

**학부 작품경진대회 은상**

### JH Agent OS

개인, 2026.02 ~ 매일 사용

Python, Git, 자동 검사

여러 AI 코딩 에이전트가 같은 사실과 규칙을 쓰게 하는 공유 계층. 검증을 통과한 변경만 기록하고 사람의 말과 AI의 추론을 출처로 나눠 추론이 사실이 되지 못하게 함

**동시 작업 덮어쓰기 차단, 백업 복구는 실제로 시험**

| | |
|---|---|
| 학력 | 전북대 컴퓨터공학부, 2026.02 졸업<br>**전공 3.83, 전체 3.71** / 4.5 |
| 자격 | **정보처리기사, SQLD, ADsP** |
| 수상 | 작품경진대회 은상<br>캡스톤디자인 우수상<br>사회서비스원 성과보고회 표창 |
| 현장실습 | (주)텔로스 백엔드, 2025.07 ~ 08<br>JWT 로그인과 권한, 좌석 관리, 출입 기록 연동 |
| 기술 | Python, Java / Spring Boot, FastAPI<br>PostgreSQL, SQLite, Docker, Linux |

01

# 판매 운영 자동화, AI 운영

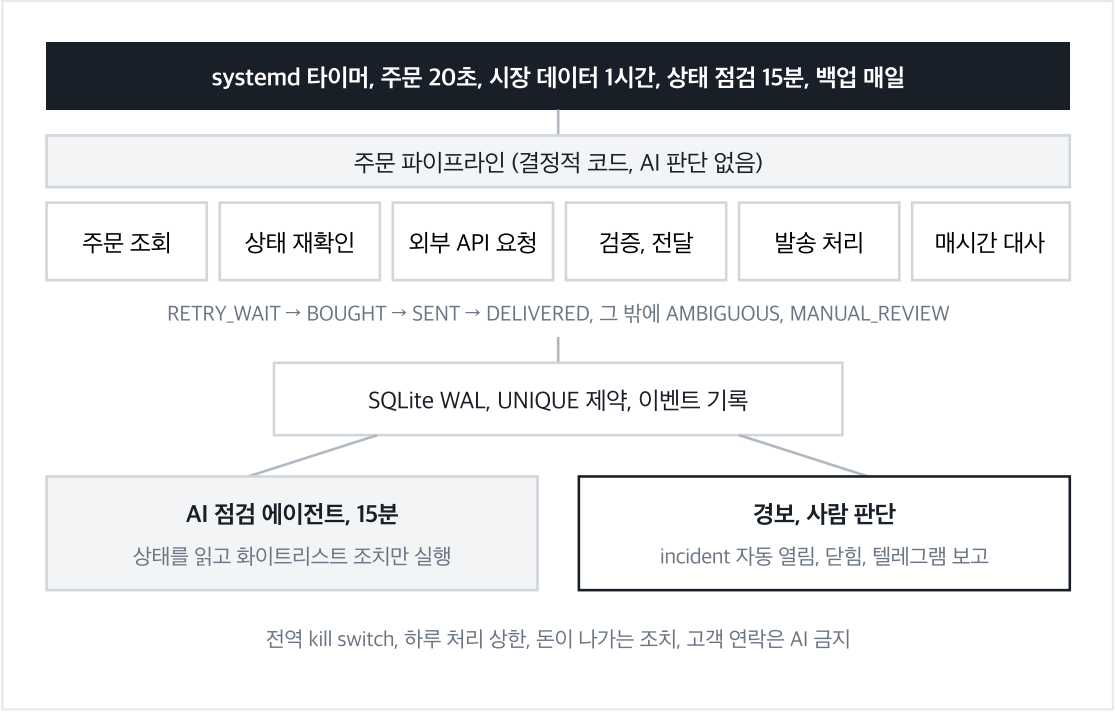
개요와 구조

개인 프로젝트, 기획부터 운영까지 직접

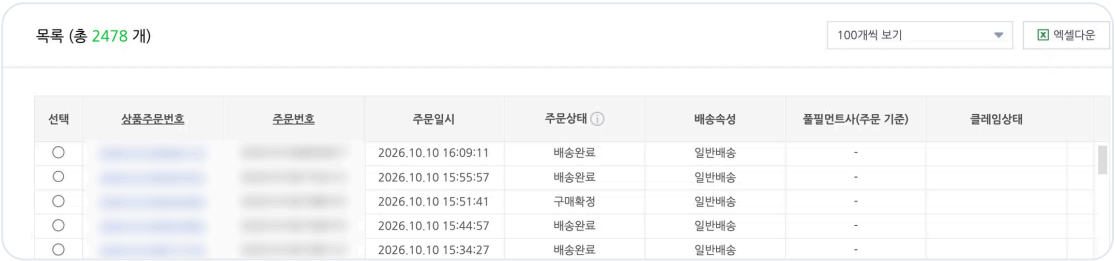
2026.09 ~ 2026.10

**직접 운영한 온라인 판매에서 공급처 발주부터 가격 광고 고객 응대 서버 관리까지 모두 혼자 맡았습니다. 하루 1,000건이 넘는 주문을 한 사람이 감당할 수 있도록 판단이 필요한 일만 사람에게 올라오고 나머지는 근거에 따라 자동으로 도는 시스템을 설계했습니다.**

| | |
|---|---|
| **Stack** | Python, SQLite(WAL), unittest |
| **Infra** | Ubuntu, systemd 타이머, 서비스 전용 계정, 매일 백업 |
| **주문, 공급** | 결제 확인 → 공급처 발주 → 고객 발송 자동 연결, 공급처 재고 하루치 미리 확보 |
| **가격, 광고** | 시간대별 가격과 광고를 바꿔 실제 이익 비교, 공개 시장 데이터 매시간 기록 |
| **고객 응대** | 상담 챗봇(버튼으로 본인 확인 후 주문 다시 받기), 문의 자동 답변, 전달 실패 감시 |
| **AI, 알림** | AI 점검 에이전트(15분, 허용된 조치만), 조치가 필요한 것만 Telegram 알림 |
| **코드** | 핵심 로직 공개 예제 github.com/qowlgur121/order-fulfillment-guard |



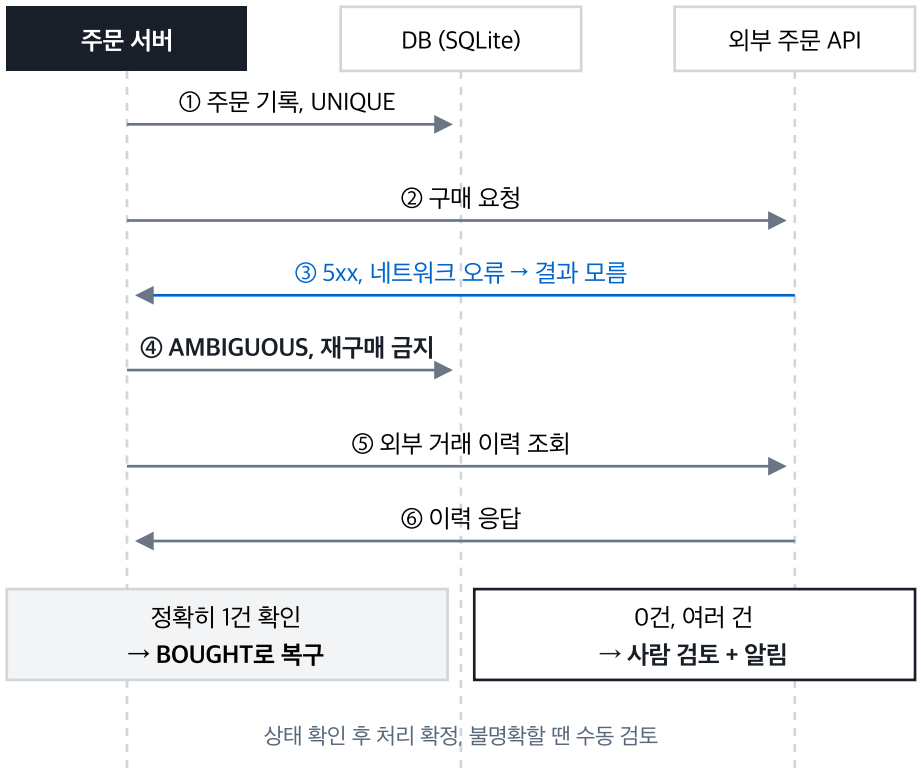

시스템 구조. 회색 상자는 AI 에이전트, 테두리 상자는 사람 판단




실제 주문 처리 목록 (주문번호는 가림)

# 외부 API 응답이 끊겨도 같은 주문이 두 번 처리되지 않게 했습니다

문제 해결 1




구매 결과를 모를 때의 처리 순서

**문제**

외부 주문 API의 응답이 끊기면 처리됐는지 알 수 없었고, 그냥 재시도하면 같은 주문이 두 번 처리될 수 있었습니다.

**설계 판단**

| | |
|---|---|
| **자동 재시도** | 동일 요청이 중복 실행될 수 있음 |
| **즉시 실패** | 실제 처리 여부와 내부 상태가 어긋날 수 있음 |
| **선택한 방식** | **상태 보류 → 거래 이력 확인 → 결과 확정** |

결과가 불명확하면 **AMBIGUOUS**로 보류하고, 외부 거래 이력에서 정확히 1건이 맞을 때만 복구했습니다. 0건이나 여러 건이면 사람 확인으로 넘겼습니다.

**결과**

불명확한 주문을 재실행하지 않고 **대조 후 확정**하는 순서로 하루 최대 1,378건의 실거래를 처리했습니다. 발송 직전에도 같은 상품 코드가 다른 주문에 나간 적이 있는지 한 번 더 확인하고, 겹치면 보내지 않고 멈추게 했습니다.

운영 환경: 실거래 피크 시간당 201건(10-07 23시). 이 규모에서는 단일 서버, 단일 writer(SQLite WAL)로 두고, 여러 인스턴스 운영, 쓰기 경합, 계측 한계 중 하나가 생기면 PostgreSQL로 옮기는 승격 조건을 설계 문서에 정했습니다
핵심 로직 예제, github.com/qowlgur121/order-fulfillment-guard

# 같은 코드를 두 처리가 동시에 잡아도 한쪽만 성공하게 했습니다

문제 해결 2



**① 같은 주문이 두 번 들어와도 한 건만**

```
UNIQUE INDEX orders(channel, channel_order_id)
```

20초마다 조회해 같은 주문을 또 읽어도 새로 생기지 않음

**② 같은 상품 코드는 한 번만 등록**

```
artifacts.fingerprint UNIQUE
```

공급처가 같은 코드를 두 번 돌려줘도 두 번째는 거부

**③ 한 코드는 한 주문에만 (조건부 UPDATE)**

```
UPDATE artifacts SET state='RESERVED',
reserved_order_id=?
WHERE id=? AND state='VALIDATED'
AND reserved_order_id IS NULL;
```

바뀐 행이 1개가 아니면 예외 → 트랜잭션 전체 되돌림

**상태 변화는 events 표에 모두 기록**

어느 주문에 어떤 코드가 언제 묶였는지 추적 가능

중복을 DB가 마지막에 막는 구조

### 문제

주문 조회는 20초마다, 점검은 15분마다 따로 돌았습니다. 같은 주문을 두 번 읽거나, 두 처리가 같은 상품 코드를 동시에 잡으면 **한 코드가 두 고객에게 가거나 같은 주문에 코드가 두 번** 나갈 수 있었습니다.

### 설계 판단

| | |
|---|---|
| **코드에서 먼저 조회 후 저장** | 조회와 저장 사이에 다른 처리가 끼어들 수 있음 |
| **별도 잠금 서버, 큐** | 서버 1대 규모에 운영할 것만 늘어남 |
| **선택한 방식** | **DB 제약 + 조건부 UPDATE로 DB가 마지막에 막게** |

SQLite를 WAL 모드로 두고, 코드 할당과 주문 갱신을 **한 트랜잭션**으로 묶었습니다. 발송 기록 저장이 실패하면 코드 할당까지 함께 되돌립니다.

### 결과

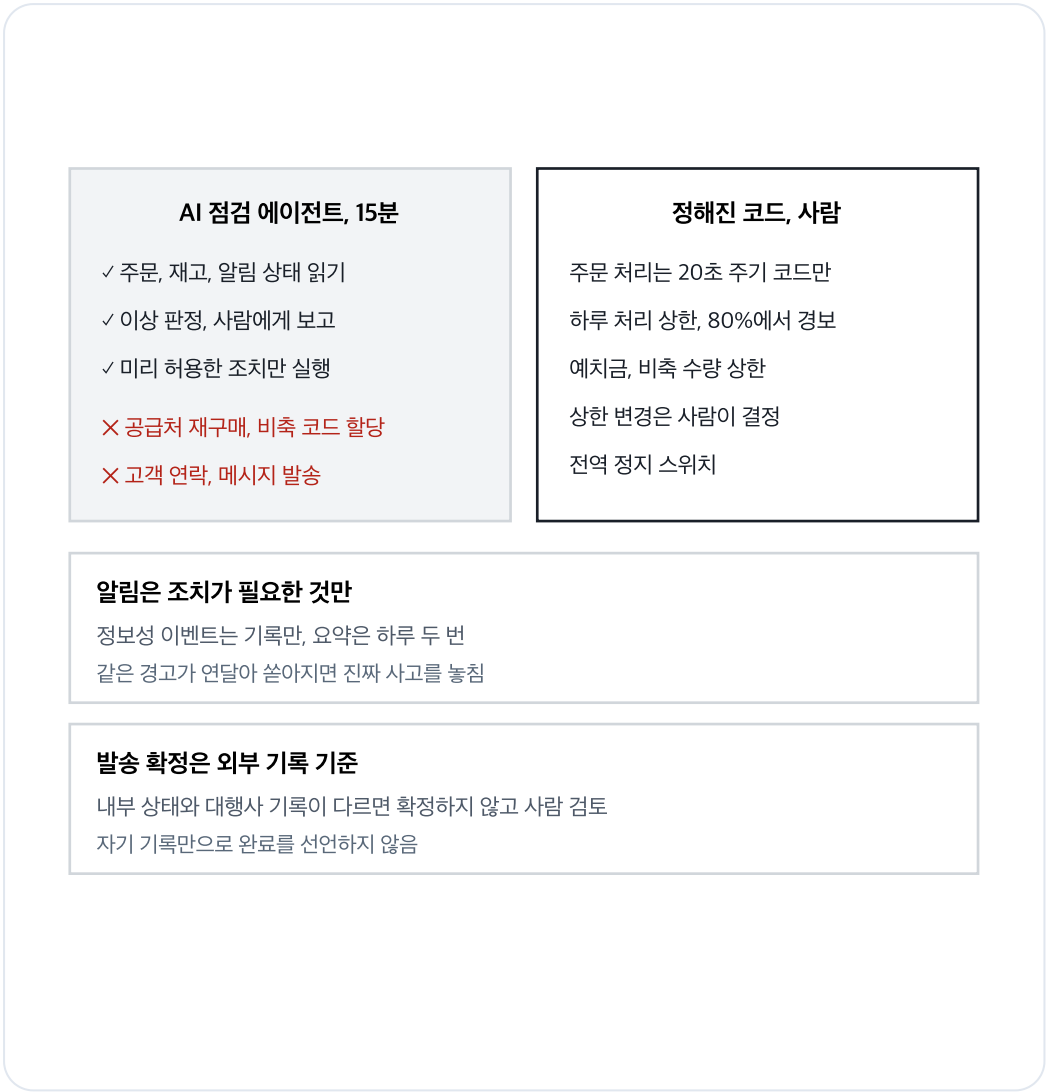
두 처리가 같은 코드를 동시에 잡아도 **한쪽만 성공**하고 나머지는 예외로 멈춥니다. 중복 코드 거부, 원자 할당, 실패 시 되돌림을 각각 테스트로 고정했습니다.

재현 테스트: github.com/qowlgur121/order-fulfillment-guard, tests/test_worker.py

# AI 에이전트에게 감시는 맡기고, 되돌릴 수 없는 권한은 주지 않았습니다

문제 해결 3






AI 에이전트, 코드, 사람의 역할 나누기

**문제**

혼자 24시간 지켜볼 수 없어 AI 점검 에이전트를 붙였습니다. 하지만 돈이 걸린 일이라, 에이전트가 잘못 판단하면 **되돌릴 수 없는 손해**가 납니다.

**설계 판단**

| | |
|---|---|
| **에이전트가 처리까지** | 같은 상황에서도 매번 다르게 판단할 수 있음 |
| **사람이 모든 주문 확인** | 하루 1,000건 넘게 오면 감당 불가 |
| **선택한 방식** | **처리는 코드, 감시는 에이전트, 돈과 한도는 사람** |

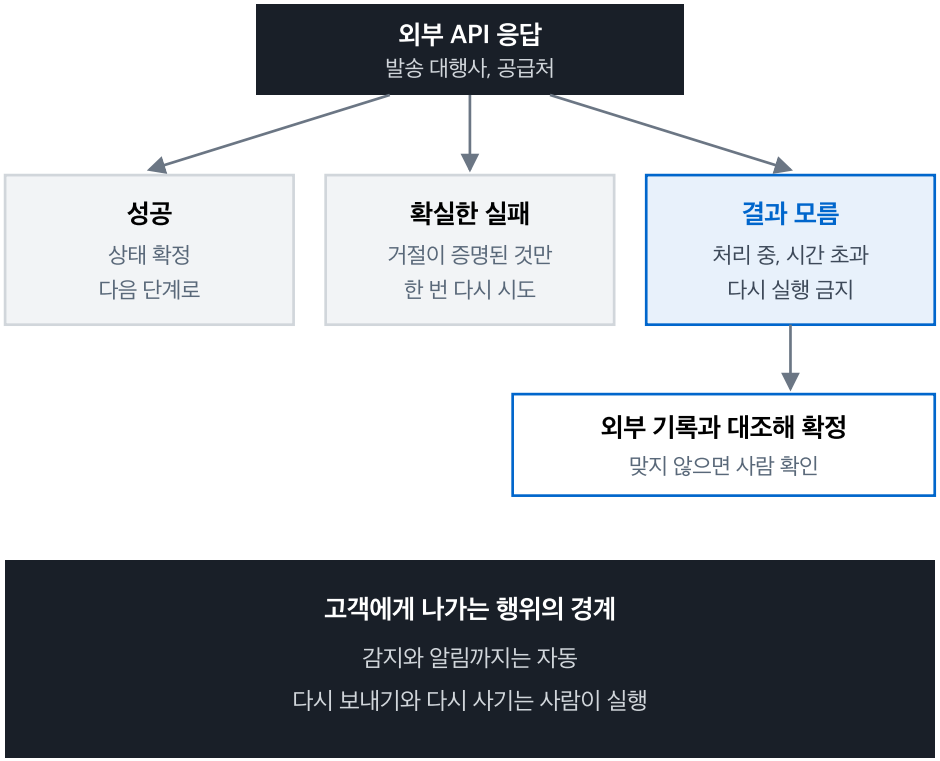
에이전트가 할 수 있는 조치를 허용 목록으로 정해 재구매와 코드 할당과 고객 발송을 막고, 규칙으로 되는 일에는 AI를 부르지 않았습니다.

**결과**

에이전트는 상태를 읽고 보고하며 가격과 광고는 미리 정한 범위 안에서만 조정하고, **재구매, 코드 할당, 고객 발송처럼 되돌릴 수 없는 조치는 정해진 코드 규칙과 제 결정으로만** 실행됩니다. 처리 상한은 정상 판매가 확인될 때만 늘어나게 했습니다.

# 외부 시스템의 '모름'을 실패로 처리하지 않고, 고객에게 나가는 자동화를 멈췄습니다

문제 해결 4




외부 결과를 셋으로 나누고, 고객에게 나가는 행위는 사람이 실행

### 문제

외부 API는 성공과 실패 말고 **아직 모름**을 돌려줄 때가 있습니다. 이것을 실패로 읽으면 같은 것을 다시 보내고, 확실한 실패를 모름으로 읽으면 처리해야 할 주문이 멈춥니다. 둘 다 실제 판매 중에 일어났습니다.

| | |
|---|---|
| **중간 상태 오판** | 감시 작업이 발송 조회 결과 '처리 중'을 '도달 실패'로 판정해 사람 승인 없이 재발송을 반복하는 것을 발견했습니다 → **자동 재발송을 코드에서 없애고**, 오래 처리 중인 건은 알림만 하게 바꿨습니다 |
| **확실한 거절 누락** | 발송 대행사의 확실한 거절까지 '결과 모름'으로 처리돼 결제된 주문이 사람 검토 상태에 머무는 것을 발견했습니다 → 확실한 거절과 모름을 코드에서 나누고, **다시 사거나 다시 보내지 않고 빠져나오는 복구 경로**를 만들었습니다 |

### 결과

외부 결과를 **성공, 확실한 실패, 결과 모름** 셋으로 나누고, 모름은 다시 실행하지 않고 외부 기록과 대조해 확정하게 했습니다. 고객에게 나가는 행위는 감지와 알림까지만 자동으로 두었습니다. 두 사고는 같은 상황을 재현하는 회귀 테스트로 남겼습니다.

# 주문과 고객을 시각으로 짝짓던 방식을 버리고, 확인된 식별 정보로만 연결했습니다

문제 해결 5




시각 추정을 없애고 확인된 식별 정보로만 연결

### 문제

결제한 고객이 상담 채팅으로도 구매 상품을 바로 받게 하는 기능이었고, 전달은 **다른 사람에게 한 번 가면 되돌릴 수 없었습니다.** 처음 구현은 채팅 동의 시각과 주문 시각을 비교해 고객을 짝지었고 단위 테스트도 통과했습니다. 그러나 실제 이벤트 순서로 직접 시험 구매해 보니 동의 신호가 주문보다 약 30초 먼저 도착했고, 지난 주문 상품까지 함께 전달됐습니다.

### 설계 판단

| | |
|---|---|
| **시간 범위 조정** | 범위를 좁혀도 같은 초에 주문이 몰리면 다른 고객과 연결될 수 있음 |
| **사람이 매번 확인** | 하루 1,000건 넘는 주문에서 감당할 수 없음 |
| **선택한 방식** | **시각 추정 제거 → 확인된 식별 정보로만 연결** |

전화번호는 원문 대신 **해시로만 저장**하고, 번호가 확인되지 않은 처음 온 고객은 버튼으로 본인 확인을 거치게 했습니다. 실제 이벤트 순서대로 재현하는 테스트를 먼저 넣고 고쳤습니다.

### 결과

두 고객이 **같은 초에 주문하는 상황을 테스트로 재현**해 각자 자기 주문만 받는 것을 확인했습니다. 기존 주문 2,106건을 미리 색인해 지난 구매 조회를 약 0.5초로 줄였습니다.

02

# 공공기관 AI 상담 챗봇, RAG

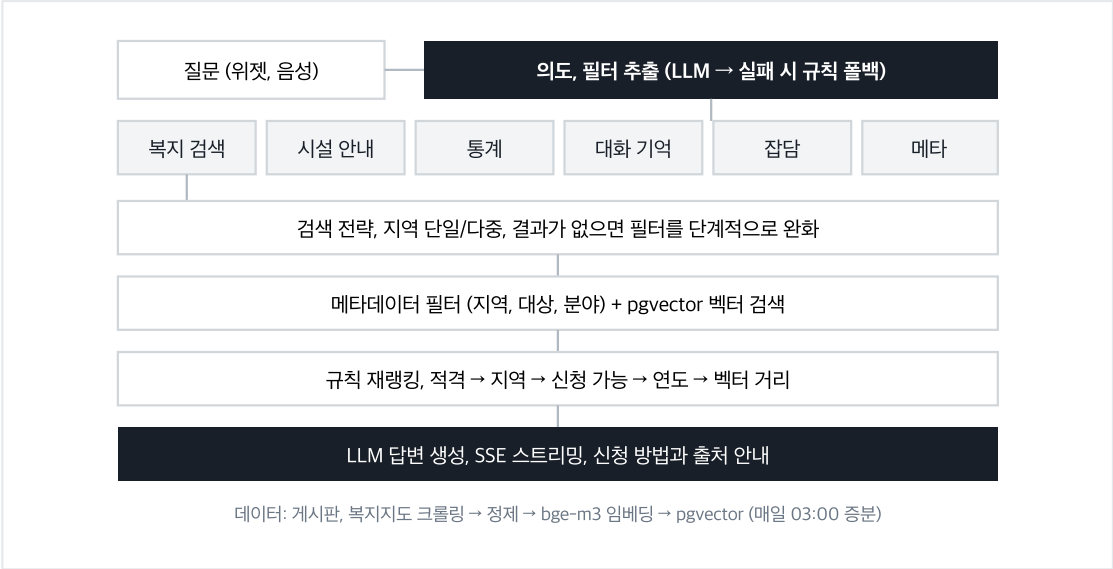
개요와 구조

산학 프로젝트, 3인 팀 팀장

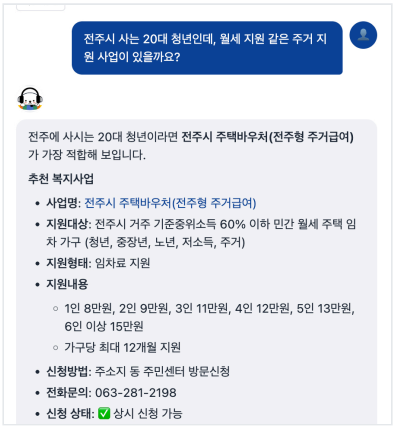
2025.09.22 ~ 2025.12.05

**기관의 기존 챗봇은 학습한 Q&A만 답해서, 새 복지사업을 안내하지 못하고 표현이 조금만 달라도 답을 못 찾았습니다. 원본 시스템은 고칠 수 없었고 운영 담당자는 개발자가 아니었습니다. 이 조건 안에서 스스로 매일 갱신되는 RAG 챗봇으로 다시 만들었습니다.**

| | |
|---|---|
| **협력** | 전북특별자치도사회서비스원 |
| **Backend** | FastAPI, SSE 스트리밍, APScheduler, Docker Compose |
| **AI, 검색** | LangChain, bge-m3 임베딩, PostgreSQL + pgvector |
| **역할** | **내 담당** 요구사항 협의, 설계, 서버, 검색, 인증<br>**팀원** 자료 조사, 발표, 문서 |




질문 하나가 답이 되기까지의 흐름



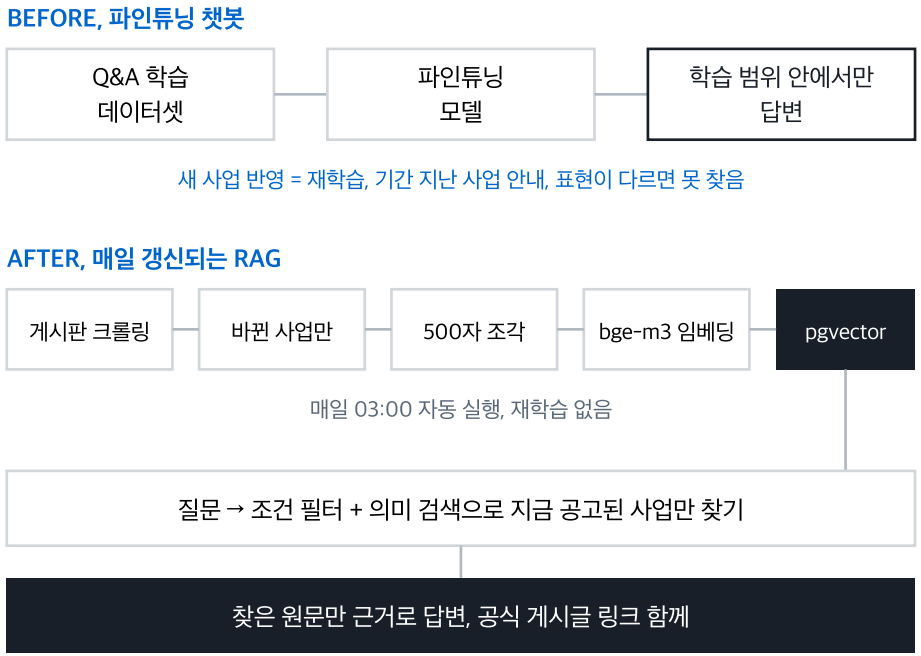




조건을 말하면 받을 수 있는 사업과 신청 방법 안내 / 증상을 말하면 가까운 의료기관을 거리순 안내

# 재학습 없이 매일 새 사업이 반영되는 RAG 구조로 바꿨습니다

문제 해결 6




기존 방식과 새 방식 비교

### 문제

기존 챗봇은 파인튜닝 기반이라 학습한 Q&A 안에서만 답했고, 새 사업을 넣으려면 다시 학습해야 해서 신청 기간이 지난 사업을 안내하기도 했습니다.

### 설계 판단, 기관 조건에서 출발

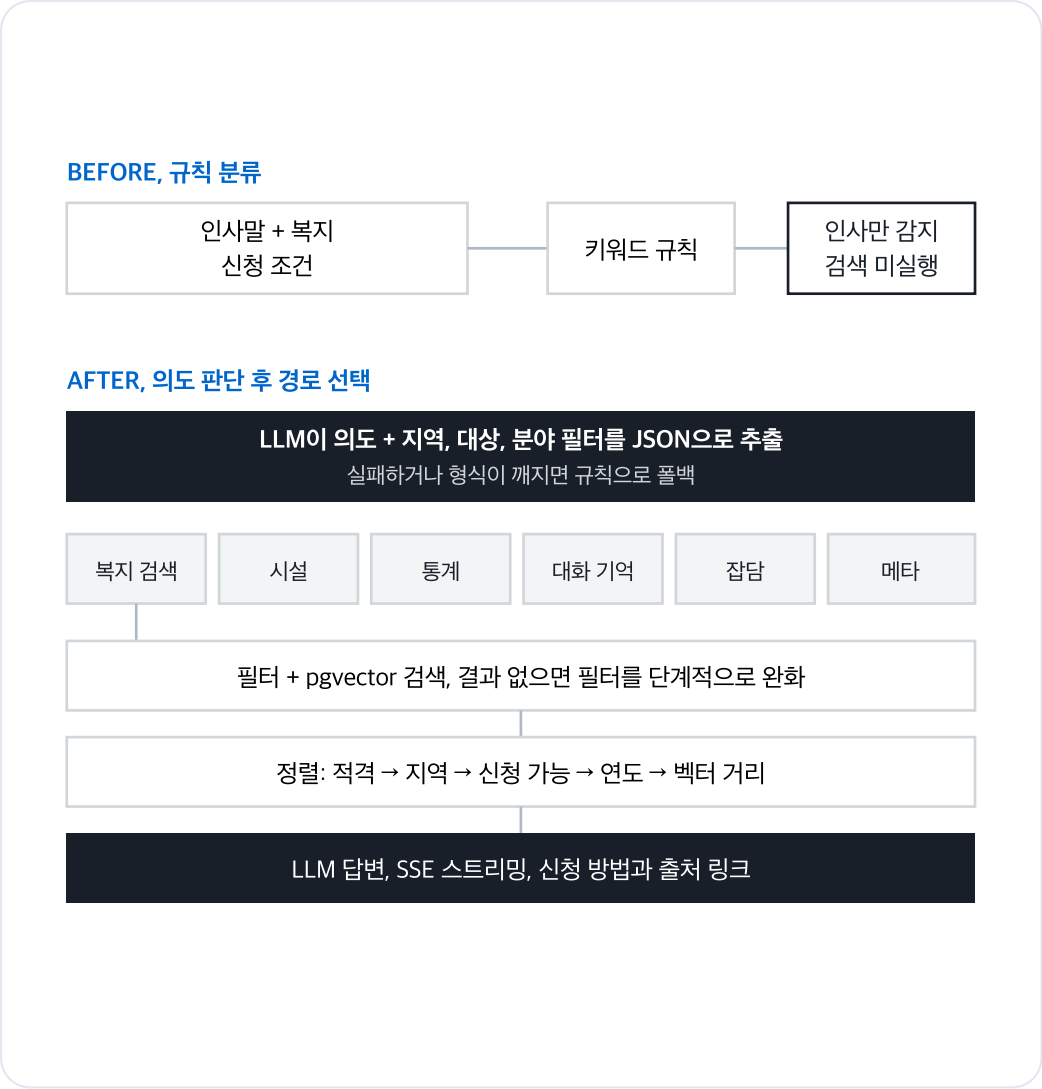
| | |
|---|---|
| **데이터 확보** | 연동 API, DB 계정 요청이 비용 문제로 거절됨 → 공개 게시판 11종을 직접 수집, 바뀐 사업만 매일 03:00 반영 |
| **서버 언어** | Spring은 공공기관 전자정부 프레임워크 부담 → 수집, 검색, LLM을 Python(FastAPI) 하나로 |
| **벡터 저장소** | 운영 담당자가 개발자가 아님 → 전용 벡터 DB 대신 PostgreSQL + pgvector로 조건 필터와 의미 검색을 한 쿼리에 |
| **기존 사이트** | **원본 수정 권한 없음 → 스크립트 한 줄로 붙는 위젯(Custom Element)** |

### 결과

재학습 없이 복지사업 1,809건이 매일 새벽 반영되고, 찾은 원문과 공식 게시글 링크로 답합니다. 관리자 화면과 운영 문서를 함께 넘겨 개발자가 없는 기관이 직접 운영할 수 있게 했습니다.

# LLM이 의도와 자격 조건을 뽑고, 신청 가능한 사업부터 답하게 했습니다

문제 해결 7




질문 처리 흐름 개선 전후

### 문제

나이와 장애 여부와 사는 지역이 한 문장에 섞인 질문은 키워드 규칙으로 처리할 수 없었고, 검색해도 기간이 지났거나 지역이 다른 사업이 먼저 나왔습니다.

### 설계 판단

| | |
|---|---|
| **규칙만 추가** | 표현이 달라질 때마다 예외 유지보수 필요 |
| **LLM 단독** | 응답 실패, 형식 오류에 대한 별도 처리 필요 |
| **선택한 방식** | **LLM으로 의도 추출 + 검증 후 규칙 폴백** |

LLM이 의도, 지역, 대상, 분야를 JSON으로 뽑아 6개 경로로 보내고, 형식이 깨지면 규칙 분류로 넘어갑니다. 결과는 **자격 → 지역 → 신청 가능 → 연도 → 벡터 거리** 순으로 다시 정렬했습니다.

### 결과

질문은 6가지 의도 경로로 나뉘고, 조건이 섞인 질문도 복지 검색으로 이어집니다. LLM이 실패하거나 형식이 깨져도 규칙으로 답해 서비스가 멈추지 않습니다.

03

# OpenContext, 사내 문서 AI 검색 MCP 서버

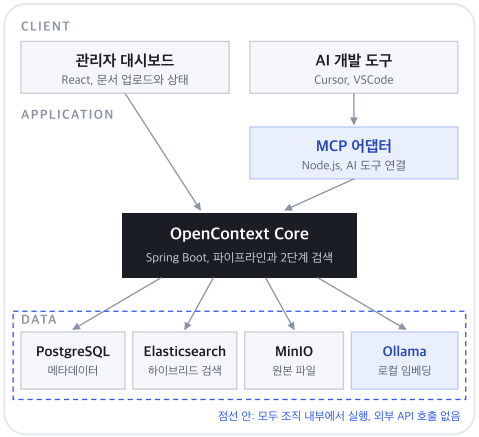
개요와 구조

3인 팀, 오픈소스 개발자대회

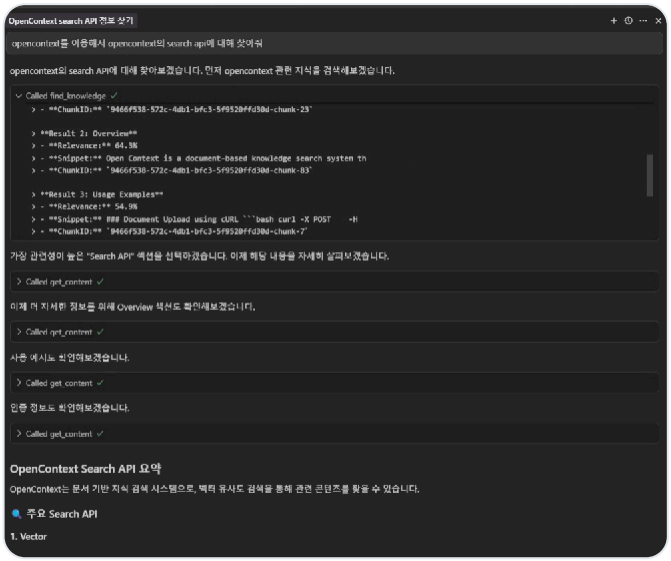
2025.08 ~ 2025.12, github.com/OpenContextAI/open-context

**현장실습에서 AI 코딩 도구에 외부 문서 검색 서비스를 붙여 보니 효과는 컸지만, 회사 문서는 밖으로 보낼 수 없어 정작 사내 자료에는 쓸 수 없었습니다. 그래서 회사가 자기 서버에 설치해 쓰는 MCP 검색 서버를 팀에 제안하고, 기획과 설계를 맡아 3인 팀으로 만들었습니다.**

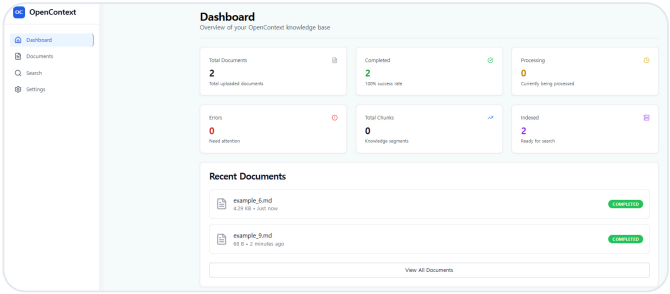
| | |
|---|---|
| **Backend** | Java 21, Spring Boot 3.3, PostgreSQL |
| **검색** | Elasticsearch(BM25), Ollama bge-m3 로컬 임베딩 |
| **연동** | MCP, Cursor, VS Code |
| **역할** | **내 담당** 제안, 기획(PRD), 설계, 하이브리드 검색 엔진, 2단계 조회 API, DB, 배포<br>**팀원** 문서 수집 파이프라인, MCP 어댑터 |




전체 구조. 점선 안은 모두 조직 내부에서 실행



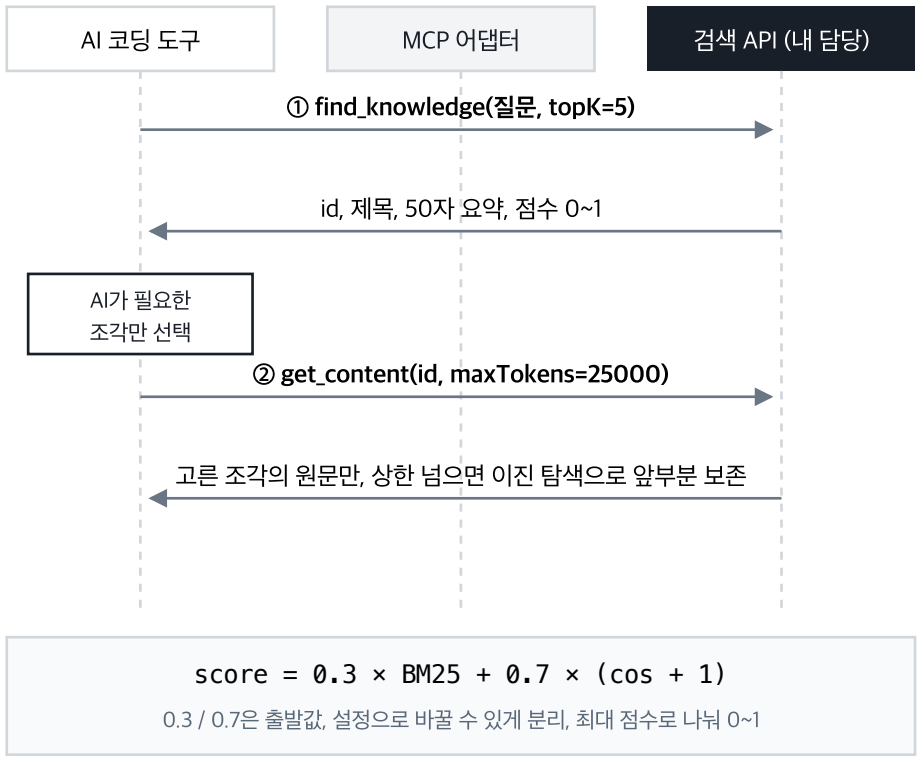

AI 도구가 find_knowledge로 고르고 get_content로 원문만 가져옴




관리자 화면, 문서 처리 상태

# AI가 목록을 먼저 보고 필요한 원문만 가져가는 2단계 검색 API

문제 해결 8




2단계 검색 호출 순서와 점수 계산

**문제**

사내 문서는 API 이름처럼 정확한 용어로도, 전혀 다른 표현으로도 찾습니다. 그리고 검색한 원문을 한 번에 다 넘기면 컨텍스트가 커져 AI가 핵심을 놓쳤습니다.

**설계 판단**

| | |
|---|---|
| **원문 전부 전달** | 컨텍스트가 커지고 AI가 쓸 부분을 못 고름 |
| **요약만 전달** | 답에 필요한 원문 근거가 빠짐 |
| **선택한 방식** | **목록 먼저 → AI가 고른 조각만 원문으로** |

**find_knowledge**가 제목, 요약, 관련도를 먼저 주고, **get_content**로 고른 원문만 가져갑니다. 정확한 용어는 BM25가, 다른 표현은 벡터 유사도가 잘 찾아서 두 점수를 합쳐 0~1로 돌려줬습니다.

**결과**

AI 도구가 **먼저 고르고 필요한 원문만 읽는 검색 경로**를 만들었습니다. 문서 처리와 임베딩은 모두 조직 안에서 돕니다.

코드: github.com/OpenContextAI/open-context — core/.../SearchService.java, ContentRetrievalService.java, SearchController.java

04

# JH Agent OS, 여러 AI 코딩 에이전트가 같은 정본과 규칙을 쓰게 하는 공유 계층

개요와 설계

개인, 직접 설계, 매일 사용

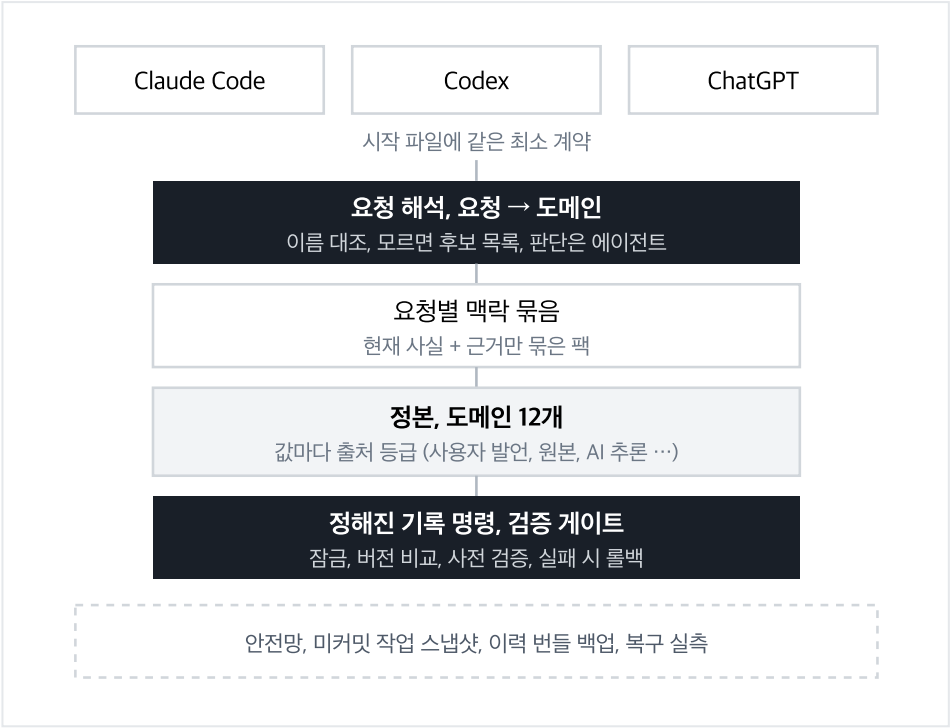
2026.02 ~ 현재

**Claude Code, Codex 같은 AI 코딩 에이전트를 여러 개 동시에 쓰면 각자 다른 파일을 읽고 같은 규칙을 따로 기억하며 서로의 작업을 덮어씁니다. 에이전트를 만드는 대신 에이전트들이 공유하는 사실, 규칙, 쓰기 경로를 설계했습니다.**

**설계 결정과 이유**

| | |
|---|---|
| **정본 하나** | 현재 사실은 도메인마다 한 곳에만 두고 나머지는 다시 만들 수 있는 파생물로. 복제된 사실은 결국 갈라진다 |
| **요청별 맥락 묶음** | 요청마다 필요한 사실과 근거만 규칙으로 골라 묶어 전달. 무엇을 넣었는지 사람이 읽고 고칠 수 있게 |
| **버전 확인 쓰기** | 도메인 잠금 + 이전 버전 해시 비교 + 사전 검증 통과 시에만 기록, 실패하면 되돌림. 동시 세션의 조용한 덮어쓰기 차단 |
| **출처 등급** | 사용자 발언, 원본 관측, 외부 관측, AI 추론 등 8종으로 기록, AI 추론은 현재 사실이 될 수 없음. 추론이 사실로 굳는 것을 코드로 막음 |
| **최소 계약 주입** | 모든 에이전트 도구의 시작 파일에 같은 계약을 직접 넣고, 다시 돌려도 바뀌지 않게. 문서를 가리키기만 하면 에이전트가 읽지 않았다 |

에이전트 실행 환경은 기존 도구를 그대로 쓰고, 그 **아래에서 모든 에이전트가 같은 사실을 보게 하는 층**을 만들었습니다.




요청이 들어와서 기록되기까지

| | |
|---|---|
| **검증** | 규칙이 빠지거나 값이 어긋나면 기록 전에 실패 |
| **복구** | 이력 전체를 번들로 백업하고 실제 복구로 확인 |
| **Stack** | Python, Git, JSON |